PyOmniGraph - Getting and hosting your own federated copies of subgraphs of Wikidata and other knowledge graphs*

Wolfgang Fahl[1,*], Tim Holzheim[1], Christoph Lange[2] and Stefan Decker[1]

[1]RWTH Aachen University, Computer Science i5, Aachen, Germany
[2]Fraunhofer FIT, Sankt Augustin, Germany

Abstract
Wikidata is an example of a very large, crowd-sourced general knowledge graph backed by a world wide community since 2012. FactGrid has been using the same Wikibase infrastructure as Wikidata since 2017 for historical research projects. GOV (Geschichtliches/Genealogisches Ortsverzeichnis) is an online gazetteer for searching ancient places. Wikidata grew so large that the Blazegraph endpoint powering it was pushed to the 4 terabyte limit and the Wikimedia Foundation therefore decided to split the knowledge graph to host two instances. The challenges of federated queries that came up in the above use cases led to the need of being able to work with subgraphs of the RDF graphs and try out different alternative endpoints such as QLever, Virtuoso and others. While typical research projects that do such subgraphing and benchmarks roll their own environments, our pyomnigraph, a Python-based open source library, provides a unified infrastructure with a standard set of REST APIs and CLI commands to improve the handling and potentially grow the matrix of subgraphs versus endpoints to work with. In this work we report on pyomnigraph usage on a matrix of three RDF graphs: Wikidata, FactGrid and GOV, and the Blazegraph, Jena, QLever endpoints on a selection of subgraph combinations. We show that typical frustrations of scaling multi-hop federated queries can be avoided if the subgraphs are locally loaded and federation is applied locally.

Keywords
Wikidata, FactGrid, QLever, Blazegraph, Apache Jena, Oxigraph, GOV, RDF triple store, federated SPARQL, benchmark, PyOmniGraph

1. Introduction

Wikidata [1] started in 2012 as an effort to back the different multilingual Wikipedia sites with a common set of facts. Initially, about 2 million entities were harvested from existing Wikipedia content. As of 2026, over 100 million entities have been curated by a growing worldwide community of contributors. In 2022, we reported on How to get and host your own copy of Wikidata [2]. For quite a few use cases, the need for federated queries based on subgraphs of diverse public and internal knowledge graphs has been our motivation to try out different combinations of endpoints and subgraphs. In the case of Wikidata we have even been forced to do so by the 2025 graph split [3].

Public alternatives to the official Wikidata Query Service WDQS are still rare. Our attempts to convince the Wikimedia Foundation to support a public Mirror infrastructure have so far been in vain. Table 1 lists the seven working public Wikidata endpoints currently configured in nicescholia[1] [4] to be watched as potential alternative Scholia backends.

The Wikimedia Foundation itself now serves three: the legacy full graph[2] and, since the WDQS graph

6th Wikidata Workshop 2026, co-located with ISWC 2026, October 25–26, 2026, Bari, Italy

*Preprint. Intended for submission to the 6th Wikidata Workshop 2026 (co-located with ISWC 2026); if accepted, to appear in the CEUR-WS proceedings. This version has not been peer-reviewed.

*Corresponding author.
wf@bitplan.com (W. Fahl)
0000-0002-0821-6995 (W. Fahl); 0000-0003-2533-6363 (T. Holzheim); 0000-0001-9879-3827 (C. Lange); 0000-0001-6324-7164 (S. Decker)

[1]https://nicescholia.wikidata.dbis.rwth-aachen.de/api/endpoints
[2]https://query-legacy-full.wikidata.org/sparql

Table 1
Public Wikidata query endpoints. Triple counts for QLever (Freiburg) and Virtuoso measured with `COUNT(*)` on 2026-07-31; the DBIS QLever count query did not return within the timeout (–).

Provider	Triple Store	Triples (B)	Last update
Wikimedia Foundation (legacy full)	Blazegraph	22.0	continuous
Wikimedia Foundation (main)	Blazegraph	8.6	continuous
Wikimedia Foundation (scholarly)	Blazegraph	8.7	continuous
RWTH Aachen DBIS	Blazegraph	17.0	2025-06-01
RWTH Aachen DBIS	QLever	–	continuous
University of Freiburg	QLever	17.7	continuous
OpenLink Software (truthy)	Virtuoso	8.1	2025-11-01

split, a main[3] and a scholarly[4] half. RWTH Aachen University hosts a full Blazegraph mirror[5] alongside a QLever endpoint[6]; the University of Freiburg QLever[7] service has moved to the `qlever.dev` host. OpenLink Software[8] provides a Virtuoso instance restricted to truthy statements. These alternatives still have their own limitations regarding SPARQL language coverage, data currency and completeness.

Using federated queries on participants of the Linked Open Data Cloud LOD Cloud[9] is in theory an excellent way to answer queries over diverse data sources. In practice there are a lot of possible frustrations involved such as outages of endpoints, incompatibilities of endpoints, awkward syntax and general query rot.

For real life use cases only that operate on small subgraphs it should be feasible to operate locally by providing the necessary data "on the fly". Unfortunately to do so a lot of technical expertise is needed to set up endpoints, collect the subgraph data and finally perform the query. To ease this pain, we introduce pyomnigraph, which mitigates the complexity by using Docker and Python to provide a unified interface to different endpoint technologies in a local environment with limited resources.

The contributions of this work are:

- **pyomnigraph**, an Apache-2.0 licensed Python library that unifies the deployment and operation of multiple RDF triple stores behind a single API, REST interface and Docker-based setup (Section 3), covering the backends listed in Section 3.1 and the `omnigraph` and `rdfdump` command-line interfaces (Section 3.2).

2. Related Work

Wikidata graph split Willighagen et al. [3] report on the three options that have been considered for surviving the 2025 Wikidata graph split: federated queries between the main and the scholar graph, using QLever [5] with modified queries on a full graph that does not suffer from the 4 TB limit of the Blazegraph endpoint, or using snapquery [6] with named parameterized queries as a middleware to separate the endpoints' concerns from the domain needs.

Linked Data Fragments The Linked Data Fragments line of work addresses the same pain point — public SPARQL endpoints can not sustain highly concurrent live query workloads — with a different infrastructure approach: smart-KG [7] ships pre-materialized, compressed graph partitions from the server to the client for client-side evaluation, and Heling and Acosta [8] provide a framework for federated SPARQL query processing over heterogeneous Linked Data Fragments.

[3]https://query-main.wikidata.org/sparql
[4]https://query-scholarly.wikidata.org/sparql
[5]https://wdqs.wikidata.dbis.rwth-aachen.de/sparql
[6]https://qlever-api.wikidata.dbis.rwth-aachen.de/
[7]https://qlever.dev/api/wikidata
[8]https://truthy.demo.openlinksw.com/sparql
[9]http://cas.lod-cloud.net/

CONSTRUCT queries CONSTRUCT queries in SPARQL have been studied by Kostylev et al. [9].

Federated SPARQL query processing Federated SPARQL query processing evaluates a single query over data that is distributed across several *remote SPARQL endpoints* [10]. What counts as a correct answer is defined by comparison with the centralised case. A federated query must return exactly the result that the same query would return over a single RDF graph holding the data of all participating endpoints [11, 12]. The difficulty is that this answer has to be produced without such a graph ever existing, and while contacting as few endpoints as possible. Two rather different mechanisms are commonly subsumed under the term, and they differ in who decides how the query is distributed. The first is part of the standard: SPARQL 1.1 provides the `SERVICE` keyword, which instructs a federated query processor to evaluate a designated part of a query against a designated remote endpoint and to combine the results it returns with those of the remaining parts [10]. Because the endpoint URLs are written into the query text, the author fixes the distribution in advance and the processor only dispatches what it is told. The second is a *federation engine*, a system layer built on top of the standard, which accepts an ordinary SPARQL query carrying no endpoint annotations, together with a catalogue of reachable endpoints, and derives the distribution itself. It does so in three stages (source selection and query decomposition, optimization, and execution), and engines differ mainly in how much knowledge the first stage draws on: zero-knowledge probing with `ASK` queries (FedX), precomputed indices or VoID summaries (SPLENDID, HiBISCuS, CostFed), or runtime adaptivity (ANAPSID) [12, 13, 14]. Throughout, the decisive optimization is the detection of *exclusive groups*, subqueries answerable by a single endpoint, which collapse many requests into one. The sources of a federation need not even be full SPARQL endpoints: interfaces of more restricted expressivity, such as triple pattern fragments, give rise to *heterogeneous* federations [12]. It is the automatic derivation of the distribution, rather than the delegation mechanism itself, that the results discussed below call into question.

Benchmarks Performance is assessed with benchmarks: FedBench has long been the reference [15], LargeRDFBench scales it to a billion triples with complex and large-data queries [14], FedShop scales the number of participating endpoints from 20 to 200 [16], QFed generates query sets dynamically for specialized systems [17], FQCEB targets cardinality estimation [18], and FKGQA extends the setting to federated question answering [19]. FedBench alone, however, cannot expose the problem at issue: all of its queries decompose into at most three subqueries, so every engine behaves alike on it [13], and with a mean of 4.3 triple patterns and runtimes of roughly two seconds its results do not extrapolate to the Data Web [14].

3. The pyomnigraph Resource

pyomnigraph [20] provides a unified Python interface for multiple graph databases. The software is open source under Apache 2.0 license and hosted on GitHub. Figure 1 shows the architecture in one panel: the `omnigraph` and `rdfdump` command line interfaces and the REST API share the unified Python API, which manages the triple store backends as Docker containers; `rdfdump` pulls subgraph dumps from public SPARQL endpoints.

pyomnigraph leverages the named parameterized query approach from the snapquery [6] framework, which handles such parameterized SPARQL queries. It supports semantic annotation of queries and monitoring of their execution. By abstracting away the underlying endpoint, it can call multiple endpoints at once to compare results, or swap endpoints transparently, without users noticing or having to change anything.

The same separation of concerns—hiding details of endpoints and queries—is used by pyomnigraph. For human readability you will find YAML versions of these details in the `examples/<usecase>/` folders of our paper repository.

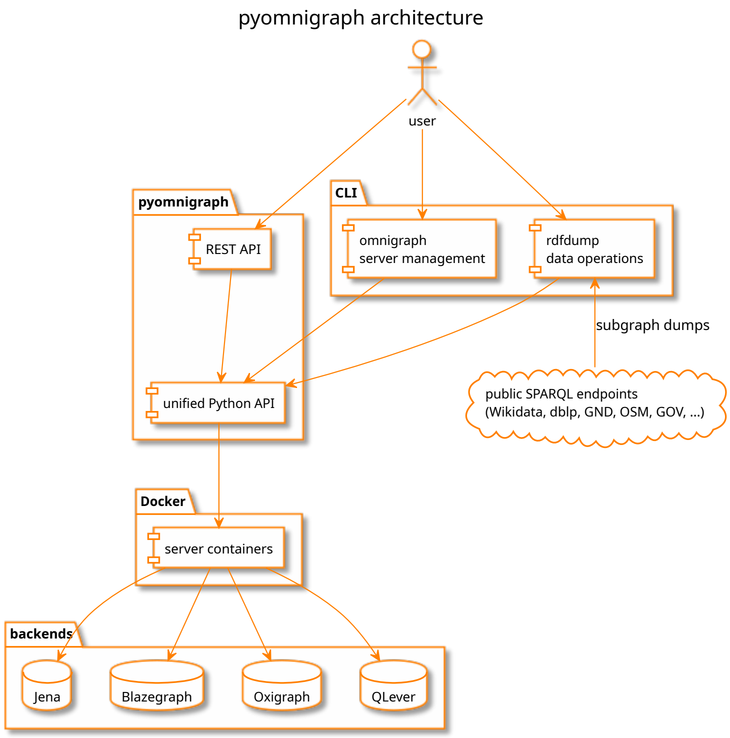

Figure 1: pyomnigraph architecture: CLI and REST API over the unified Python API, Docker managed triple store backends, subgraph dumps from public SPARQL endpoints. Source: `figures/architecture.puml` (PlantUML).

Table 2
Supported triple store backends per the pyomnigraph README as of 2026-07-31.

Database	Status	Notes
Apache Jena	working	robust, standards compliant
Blazegraph	working	high performance, easy setup
Oxigraph	working	Rust-based, embedded, fast
QLever	working	extremely fast queries
GraphDB	disabled	license required (free version available)
MillenniumDB	disabled	import-first workflow, Property Graph + RDF
Virtuoso	disabled	permissions issue
Stardog	planned	license required

3.1. Supported Backends

Table 2 reproduces the supported triple stores table of the pyomnigraph README[10].

3.2. Command-Line Interfaces

There are two command line interfaces available for pyomnigraph: `omnigraph` for server management and `rdfdump` for data operations.

4. Use cases and Data Sources

Three use cases are the basis for our demonstration and evaluation: papers and authors (scholarly), historical locations for genealogy research and railway journey planning. For each use case we specify

[10]https://github.com/WolfgangFahl/pyomnigraph

Table 3
Public SPARQL endpoints serving the subgraph sources, probed 2026-07-31 with `SELECT * WHERE {?s ?p ?o} LIMIT 1`.

Dataset	Endpoint	Engine	Probe
dblp	qlever.dev/api/dblp	QLever	0.072 s
GND	sparql.dnb.de/api/gnd	QLever	0.153 s
Wikidata	query.wikidata.org/sparql	Blazegraph	0.355 s
FactGrid	database.factgrid.de/sparql	Blazegraph	0.100 s
OpenStreetMap	sophox.org/sparql	Blazegraph	0.129 s
OpenStreetMap	qlever.dev/api/osm-planet	QLever	0.619 s
ERA RINF	rinf.data.era.europa.eu/api/sparql	RDF4J	0.139 s
GOV	gov-sparql.genealogy.net/dataset/sparql	Jena	0.060 s

a list of information needs, from which we derive concrete example queries. The abstraction is then done by applying named parameterized queries.

Table 3 shows the subgraph sources for the use cases.

Table 4 tracks the named parameterized queries that we use to demo and benchmark, one row per query and example binding. Each query is defined in `examples/<usecase>/queries.yaml` with its parameter list and default values, each endpoint in `examples/<usecase>/endpoints.yaml`, so a row is reproduced by a single `sparqlquery` [21] call; the `query.sh` script of each use case reruns all its queries and writes a `<QueryName>.md` result file per query.

The examples can be reproduced from the pyomnigraphPaper GitHub repository[11]. Result sets that exceed the page limit of this paper are imported into the repository's query issues, one issue per query; the TryQueries[12] page complements them with short try-it links that run each query interactively against its endpoint.

As typical for real world use cases, the ideal persistent identifiers for linking have limited availability. ORCID and ROR would be such identifiers for the scholarly case, according to Jesper Zedlitz, the counterpart for GOV would be a reconciliation via the current municipalities in Germany. This should work well via the Amtlicher Gemeindeschlüssel (official municipality key, P439). PID candidates for railway stations are the UIC station code P722[13], standardized by the UIC in Paris, or the international station number IBNR P954[14], which could formerly be resolved via the Deutsche Bahn departure board formatter URL[15] — marked with "link rot" in Wikidata, which is grist to our mill. The Adif page for Irún[16], as used in our ebike wiki[17], uses yet another number.

4.1. scholarly - authors and papers

Queries: the papers of an author, the papers of all coauthors of a given paper, the papers of all authors of a given scientific event, the papers of all authors of a given institution

4.2. historical locations

For the historical location gazetteer GOV [22], a List of Queries[18] has been curated, which need to be extended by references to FactGrid and Wikidata.

[11] https://github.com/WolfgangFahl/pyomnigraphPaper
[12] https://cr.bitplan.com/index.php/TryQueries
[13] https://www.wikidata.org/wiki/Property:P722
[14] https://www.wikidata.org/wiki/Property:P954
[15] https://reiseauskunft.bahn.de/bin/bhftafel.exe/en?input=$1&boardType=dep&time=actual&productsDefault=1111101&start=yes
[16] https://www.adif.es/en/w/11600-ir%C3%BAn
[17] https://ebike.bitplan.com/index.php/Irun
[18] https://gov-wiki.genealogy.net/index.php?title=List_of_Queries

Table 4
Named parameterized queries per use case, with the example parameter binding used for the reproduction run and the resulting number of records. Run with `sparqlquery` against the endpoint declared in `examples/<usecase>/endpoints.yaml`

Use case	Query	Dataset	Example	Records
scholarly	AuthorPapers	dblp	27/6858 (Fahl)	6
scholarly	CoauthorPapers	dblp	FahlHW0D22a	600
scholarly	EventAuthorPapers	Wikidata	Q133307039	235
scholarly	InstitutionAuthorPapers	GND	36225-6	27
locations	PlaceIdentity	GOV	AACHENJO30BS (Aachen)	6
locations	PlaceHierarchy	GOV	AACHENJO30BS (Aachen)	2
locations	PlaceNames	GOV	AACHENJO30BS (Aachen)	3
locations	PersonsOfPlace	Wikidata	AACHENJO30BS (Aachen)	5543
railway	RelationStops	OpenStreetMap	10492086 (MD 18061)	18
railway	LineIdentity	Wikidata	Q801991 (Bordeaux–Irun)	1
railway	StationsOfLine	ERA RINF	655000-1 (Bordeaux–Irun)	92
railway	StationLocality	OpenStreetMap	Q3095803 (Irun)	5

4.3. railway journey planning

This use case is inspired by our velorail[19] work.

5. Federated Query Demo and Benchmark

pyomnigraph is a useful tool for trying out federated queries in a local environment to avoid the typical frustrations to be expected from public endpoints. The *examples* directory of our GitHub paper repository shows how pyomnigraph's declarative approach and command line tools simplify the task.

Listing 1 shows the `load.sh` script that loads the railway use case datasets into all functional pyomnigraph backends.

Listing 1: Loading the railway use case datasets into all functional pyomnigraph backends

```bash
#!/usr/bin/env bash
# load the railway use case datasets into all functional pyomnigraph backends
# see https://github.com/WolfgangFahl/pyomnigraphPaper/issues/10
script_dir=$(dirname "$0")
for dataset in relation_stops_osm line_identity_wikidata stations_of_line_rinf
    station_locality_osm
do
  rdfdump -dc "$script_dir/datasets.yaml" -ds $dataset --dump -4o
  omnigraph -dc "$script_dir/datasets.yaml" -ds $dataset -s all --cmd upload count
done
```

Note how calling the command line tools `rdfdump` and `omnigraph` in a loop creates a whole matrix of subgraphs versus backends. The `-s all` option covers the local backends marked working in Table 2 (Apache Jena, Blazegraph, Oxigraph, QLever); the Engine column of Table 3 refers to the public source endpoints the subgraphs are dumped from.

6. Evaluation

By applying the matrix approach of Section 5 we obtain interesting results on performance and failure cases. We then do a comparison of remote-remote-remote versus local-local-local federated queries across all possible combinations.

[19]https://github.com/WolfgangFahl/velorail

7. Availability

pyomnigraph is open source under the Apache 2.0 license, developed on GitHub[20], released on PyPI[21] and archived on Zenodo [20] under the concept DOI 10.5281/zenodo.21721772; version 0.2.3 is the release this paper refers to. The reproducible examples and the paper sources live in the pyomnigraphPaper repository[22]; the underlying libraries pyLoDStorage [21] and snapquery [6] are archived on Zenodo as well.

8. Conclusion and Future Work

Declaration on Generative AI

AI assistance in this project is limited to spelling, grammar, and formatting/build tooling; AI is NOT used to generate scientific content, arguments, or prose.

References

[1] D. Vrandečić, M. Krötzsch, Wikidata: A free collaborative knowledgebase, Communications of the ACM 57 (2014) 78–85. doi:10.1145/2629489.

[2] W. Fahl, T. Holzheim, C. Lange, A. Westerinen, S. Decker, Getting and hosting your own copy of Wikidata, in: Proceedings of the 3rd Wikidata Workshop 2022, volume 3262 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2022. URL: https://ceur-ws.org/Vol-3262/paper9.pdf, wikidata Q115055567.

[3] E. L. Willighagen, D. Mietchen, P. Patel-Schneider, K. Linden, J. Kalmbach, L. G. Willighagen, W. Fahl, H. Bast, Scholia 2026: Compliance with SPARQL 1.1, in: SWAT4HCLS 2026: The 17th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences, March 23–26, 2026, Amsterdam, The Netherlands, SWAT4HCLS, 2026, pp. 1–3. URL: https://hdl.handle.net/2066/331667.

[4] W. Fahl, nicescholia, 2025. doi:10.5281/zenodo.17967779.

[5] H. Bast, B. Buchhold, QLever: A query engine for efficient SPARQL+text search, in: Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM), ACM, 2017, pp. 647–656. doi:10.1145/3132847.3132921.

[6] W. Fahl, T. Holzheim, C. Hoyt, D. Priskorn, et al., SnapQuery: FAIR Management of Query Sets to Mitigate Query Rot in Knowledge Graphs, 2026. doi:10.5281/zenodo.17802424.

[7] A. Azzam, A. Polleres, J. D. Fernández, M. Acosta, smart-KG: Partition-based linked data fragments for querying knowledge graphs, Semantic Web 15 (2024) 1791–1835. doi:10.3233/SW-243571.

[8] L. Heling, M. Acosta, A framework for federated SPARQL query processing over heterogeneous linked data fragments, CoRR abs/2102.03269 (2021). URL: https://arxiv.org/abs/2102.03269. arXiv:2102.03269.

[9] E. V. Kostylev, J. L. Reutter, M. Ugarte, CONSTRUCT queries in SPARQL, in: 18th International Conference on Database Theory (ICDT 2015), volume 31 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2015, pp. 212–229. doi:10.4230/LIPIcs.ICDT.2015.212.

[10] E. Prud'hommeaux, C. Buil-Aranda, SPARQL 1.1 Federated Query, W3C Recommendation, World Wide Web Consortium (W3C), 2013. URL: https://www.w3.org/TR/sparql11-federated-query/, w3C Recommendation 21 March 2013.

[11] J. Aimonier-Davat, B. Nédelec, M.-H. Dang, P. Molli, H. Skaf-Molli, FedUP: Querying large-scale federations of SPARQL endpoints, in: Proceedings of the ACM Web Conference 2024,

[20] https://github.com/WolfgangFahl/pyomnigraph
[21] https://pypi.org/project/pyomnigraph/
[22] https://github.com/WolfgangFahl/pyomnigraphPaper

ACM, 2024, pp. 2315–2324. URL: https://dl.acm.org/doi/10.1145/3589334.3645704. doi:10.1145/3589334.3645704.

[12] L. Heling, M. Acosta, Federated SPARQL query processing over heterogeneous linked data fragments, in: Proceedings of the ACM Web Conference 2022, ACM, 2022, pp. 1047–1057. URL: https://dl.acm.org/doi/10.1145/3485447.3511947. doi:10.1145/3485447.3511947.

[13] M.-E. Vidal, S. Castillo, M. Acosta, G. Montoya, G. Palma, On the selection of SPARQL endpoints to efficiently execute federated SPARQL queries, in: A. Hameurlain, J. Küng, R. Wagner (Eds.), Transactions on Large-Scale Data- and Knowledge-Centered Systems XXV, volume 9620, Springer Berlin Heidelberg, 2016, pp. 109–149. URL: http://link.springer.com/10.1007/978-3-662-49534-6_4. doi:10.1007/978-3-662-49534-6_4, series Title: Lecture Notes in Computer Science.

[14] M. Saleem, A. Hasnain, A.-C. Ngonga Ngomo, LargeRDFBench: A billion triples benchmark for SPARQL endpoint federation, SSRN Electronic Journal (2018). URL: https://www.ssrn.com/abstract=3199316. doi:10.2139/ssrn.3199316.

[15] G. Montoya, M.-E. Vidal, O. Corcho, E. Ruckhaus, C. Buil-Aranda, Benchmarking federated SPARQL query engines: Are existing testbeds enough?, in: P. Cudré-Mauroux, J. Heflin, E. Sirin, T. Tudorache, J. Euzenat, M. Hauswirth, J. X. Parreira, J. Hendler, G. Schreiber, A. Bernstein, E. Blomqvist (Eds.), The Semantic Web – ISWC 2012, volume 7650, Springer Berlin Heidelberg, 2012, pp. 313–324. URL: http://link.springer.com/10.1007/978-3-642-35173-0_21. doi:10.1007/978-3-642-35173-0_21, series Title: Lecture Notes in Computer Science.

[16] M.-H. Dang, J. Aimonier-Davat, P. Molli, O. Hartig, H. Skaf-Molli, Y. Le Crom, FedShop: A benchmark for testing the scalability of SPARQL federation engines, in: T. R. Payne, V. Presutti, G. Qi, M. Poveda-Villalón, G. Stoilos, L. Hollink, Z. Kaoudi, G. Cheng, J. Li (Eds.), The Semantic Web – ISWC 2023, volume 14266, Springer Nature Switzerland, 2023, pp. 285–301. URL: https://link.springer.com/10.1007/978-3-031-47243-5_16. doi:10.1007/978-3-031-47243-5_16, series Title: Lecture Notes in Computer Science.

[17] N. A. Rakhmawati, M. Saleem, S. Lalithsena, S. Decker, QFed: Query set for federated SPARQL query benchmark, in: Proceedings of the 16th International Conference on Information Integration and Web-based Applications & Services, ACM, 2014, pp. 207–211. URL: https://dl.acm.org/doi/10.1145/2684200.2684321. doi:10.1145/2684200.2684321.

[18] J. Xu, P. Tian, J. Gu, The evaluation benchmark for SP ARQL federated query, in: 2023 Asia-Pacific Conference on Image Processing, Electronics and Computers (IPEC), IEEE, 2023, pp. 447–452. URL: https://ieeexplore.ieee.org/document/10412775/. doi:10.1109/IPEC57296.2023.00083.

[19] D. Dobriy, F. Bauer, A. Azzam, D. Banerjee, A. Polleres, Agentic SPARQL: Evaluating SPARQL-MCP-powered intelligent agents on the federated KGQA benchmark, 2026. URL: https://research.wu.ac.at/en/publications/83c86964-2d48-46f1-b655-5bef78c1a837. doi:10.57938/83C86964-2D48-46F1-B655-5BEF78C1A837, artwork Size: 10 pages Medium: application/pdf Pages: 10 pages.

[20] W. Fahl, pyomnigraph: Unified Python interface for multiple graph databases, https://github.com/WolfgangFahl/pyomnigraph, 2026. doi:10.5281/zenodo.21721772, apache-2.0, version 0.2.3.

[21] W. Fahl, T. Holzheim, pyLoDStorage, 2026. doi:10.5281/zenodo.17805335.

[22] J. Zedlitz, N. Luttenberger, Modelling (historical) administrative information on the semantic web, in: WEB 2014: The Second International Conference on Building and Exploring Web Based Environments, April 20–24, 2014, Chamonix, France, IARIA, 2014, pp. 33–39. URL: https://www.thinkmind.org/index.php?view=article&articleid=web_2014_2_30_40037.